macserialjunkie.com

[INFO] MIP-macOS Injection Platform - Page 6

Post by jdawg » Sat Jun 07, 2025 7:32 am

13–16 minutes

🚀 MIP - macOS Injection Platform Tutorial

MIP - macOS Injection Platform Tutorial - A comprehensive beginner's guide to code injection on macOS.

📚 Table of Contents

1. What is MIP?
2. Key Terms for Beginners
3. Prerequisites
4. Step 1: Disable SIP
5. Step 2: Install Prerequisites
6. Step 3: Download and Compile MIP
7. Step 4: Understanding MIP Architecture
8. Step 5: Creating Your First Bundle
9. Step 6: Compile and Install Your Bundle
10. Step 7: Advanced Examples
11. Step 8: Debugging and Troubleshooting

12. Step 9: Best Practices
13. Step 10: Uninstalling MIP

▶ What is MIP?

MIP (macOS Injection Platform) is a platform that lets macOS developers create operating system tweaks by allowing them to create code that can be injected automatically to any GUI process. Think of it as a way to modify how applications behave on your Mac by inserting your own code into them.

▶ Key Terms for Beginners

🔧 Essential Concepts

Code Injection
The process of inserting your own code into a running application to modify its behavior.

Dynamic Library (dylib)
A file containing code that can be loaded and used by applications at runtime. On macOS, these have a .dylib extension.

SIP (System Integrity Protection)
Prevents system files modification and debugging of SIP-protected binaries. You'll need to disable this to use MIP.

Bundle
A package containing code and resources that MIP can inject into applications.

Mach-O
The file format used for executables and libraries on macOS.

ARM64/Apple Silicon
The newer Mac processors (M1, M2, M3, etc.) that use ARM

architecture instead of Intel x86.

▶ Prerequisites

🖥️ System Requirements

- macOS Yosemite (10.10) or newer
- Xcode's command-line tools, as well as binutils for gobjcopy (brew install binutils)
- A code signing certificate (can be self-signed)
- SIP must be disabled (we'll cover this)

🔧 For Apple Silicon Macs

- On ARM64 Macs, you will also need to enable the arm64e preview ABI
- Command: sudo nvram boot-args=-arm64e_preview_abi

🔧 For Intel Macs

- On Intel Macs, you will need the 10.13 SDK, to compile the 32-bit portions of MIP

1️⃣ Step 1: Disable SIP (System Integrity Protection)

⚠️ WARNING ⚠️
Disabling SIP reduces your system's security. Only do this on a development machine. n00bs may worry - reverse engineers and crackers usually dont care about this 😀

📋 Instructions:

1. Restart your Mac and hold Command + R during boot to enter Recovery Mode
2. Open Terminal from the Utilities menu
3. Run: csrutil disable

4. Restart your Mac normally
5. Verify SIP is disabled: csrutil status (should show "disabled")

🍎 For Apple Silicon Macs (Additional Step)

Code: Select all

```
sudo nvram boot-args=-arm64e_preview_abi
```

2️⃣ Step 2: Install Prerequisites

🛠️ Install Xcode Command Line Tools

🍺 Install Homebrew (if not already installed)

Code: Select all

```
/bin/bash -c "$(curl -fsSL https://
raw.githubusercontent.com/Homebrew/install/HEAD/
install.sh)"
```

🔧 Install binutils

🔐 Create a Self-Signed Certificate

1. Open Keychain Access
2. Go to Keychain Access > Certificate Assistant > Create a Certificate
3. Name: MIP-Signer (or any name you prefer)
4. Identity Type: Self Signed Root
5. Certificate Type: Code Signing
6. Click Create

3️⃣ Step 3: Download and Compile MIP

Code: Select all

```
# Clone the repository
```

```
git clone https://github.com/LIJI32/MIP.git
cd MIP

# Compile MIP (replace "MIP-Signer" with your
certificate name)
make SIGN_IDENTITY="MIP-Signer"

# Install MIP
sudo make install
```

4️⃣ Step 4: Understanding MIP Architecture

MIP operates by injecting code to a system process (launchservicesd), which later injects code to all other processes. Here's how it works:

🔄 MIP Injection Flow:

1. Boot Process
A launch daemon runs when macOS starts

2. System Injection
Code is injected into launchservicesd

3. Application Hooking
When apps start, they call _LSApplicationCheckIn

4. Code Loading
MIP injects your bundles before the app fully launches

5️⃣ Step 5: Creating Your First Bundle

Let's create a simple "Hello World" bundle that displays a message when injected into applications.

📁 Directory Structure

Code: Select all

```
MyFirstBundle/
├── Info.plist
├── Makefile
└── main.c
```

main.c

Code: Select all

```
#include <stdio.h>
#include <syslog.h>
#include <Cocoa/Cocoa.h>

// Constructor function - runs when the bundle is
loaded
__attribute__((constructor))
static void initialize() {
    // Log to system console
    NSLog(@"🎉MyFirstBundle loaded into process:
%s",
          [[[NSProcessInfo processInfo] processName]
UTF8String]);

    // Show a simple alert (only for GUI apps)
    dispatch_async(dispatch_get_main_queue(), ^{
        NSAlert *alert = [[NSAlert alloc] init];
        [alert setMessageText:@"Hello from MIP!"];
        [alert setInformativeText:@"Your bundle was
successfully injected!"];
        [alert addButtonWithTitle:@"Cool!"];
        [alert runModal];
    });
```

```
}

// Method swizzling to intercept events
+ (void)load {
    static dispatch_once_t onceToken;
    dispatch_once(&onceToken, ^{
        Class class = [NSApplication class];

        SEL originalSelector =
@selector(sendEvent:);
        SEL swizzledSelector =
@selector(hookedSendEvent:);

        Method originalMethod =
class_getInstanceMethod(class, originalSelector);
        Method swizzledMethod =
class_getInstanceMethod(class, swizzledSelector);


method_exchangeImplementations(originalMethod,
swizzledMethod);
    });
}

- (void)hookedSendEvent:(NSEvent *)event {
    // Log all mouse clicks
    if ([event type] == NSEventTypeLeftMouseDown) {
        NSLog(@"🖱 Left mouse clicked at: %@",
NSStringFromPoint([event locationInWindow]));
    }
```

```
    // Call the original method
    [self hookedSendEvent:event];
}
```

Info.plist

Code: Select all

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>CFBundleIdentifier</key>
    <string>com.yourname.myfirstbundle</string>

    <key>CFBundleName</key>
    <string>MyFirstBundle</string>

    <key>CFBundleVersion</key>
    <string>1.0</string>

    <!-- Only inject into these apps (whitelist) -->
    <key>MIPBundleNames</key>
    <array>
        <string>com.apple.TextEdit</string>
        <string>com.apple.Safari</string>
    </array>

    <!-- Never inject into these apps -->
    <key>MIPExcludedBundleNames</key>
```

```
    <array>
        <string>com.apple.loginwindow</string>
    </array>

    <key>MIPUseBlacklistMode</key>
    <false/>

    <key>MIPSupportsGC</key>
    <false/>
</dict>
</plist>
```

Makefile

Code: Select all

```
BUNDLE_NAME = MyFirstBundle
BUNDLE_IDENTIFIER = com.yourname.myfirstbundle
SIGN_IDENTITY ?= MIP-Signer

# Compiler flags
CFLAGS = -Wall -Wextra -std=c99 -fPIC -shared
OBJCFLAGS = -fobjc-arc -framework Cocoa -framework
Foundation

# Source files
SOURCES = main.c

# Output
BUNDLE_DIR = $(BUNDLE_NAME).bundle
DYLIB = $(BUNDLE_DIR)/Contents/MacOS/$(BUNDLE_NAME)
```

```
all: $(DYLIB)

$(BUNDLE_DIR):
	mkdir -p $(BUNDLE_DIR)/Contents/MacOS
	cp Info.plist $(BUNDLE_DIR)/Contents/

$(DYLIB): $(BUNDLE_DIR) $(SOURCES)
	clang $(CFLAGS) $(OBJCFLAGS) -o $@
$(SOURCES)
	codesign -fs "$(SIGN_IDENTITY)" $@

install: $(DYLIB)
	sudo cp -r $(BUNDLE_DIR) /Library/Apple/
System/Library/Frameworks/mip/Bundles/

uninstall:
	sudo rm -rf /Library/Apple/System/Library/
Frameworks/mip/Bundles/$(BUNDLE_DIR)

clean:
	rm -rf $(BUNDLE_DIR)

.PHONY: all install uninstall clean
```

6 Step 6: Compile and Install Your Bundle

Code: Select all

```
# Navigate to your bundle directory
cd MyFirstBundle

# Compile the bundle
```

```
make SIGN_IDENTITY="MIP-Signer"

# Install the bundle
make install

# Test by opening TextEdit or Safari
open -a TextEdit
```

7️⃣ Step 7: Advanced Examples

🪟 Example 1: Window Transparency Modifier

This bundle makes all windows semi-transparent:

Code: Select all

```
#include <Cocoa/Cocoa.h>

@interface NSWindow (TransparencyHook)
- (void)originalMakeKeyAndOrderFront:(id)sender;
@end

@implementation NSWindow (TransparencyHook)

+ (void)load {
    static dispatch_once_t onceToken;
    dispatch_once(&onceToken, ^{
        Class class = [self class];

        SEL originalSelector =
@selector(makeKeyAndOrderFront:);
        SEL swizzledSelector =
@selector(hookedMakeKeyAndOrderFront:);
```

```
        Method originalMethod =
class_getInstanceMethod(class, originalSelector);
        Method swizzledMethod =
class_getInstanceMethod(class, swizzledSelector);


method_exchangeImplementations(originalMethod,
swizzledMethod);
    });
}

- (void)hookedMakeKeyAndOrderFront:(id)sender {
    [self hookedMakeKeyAndOrderFront:sender];

    // Make window 80% opaque
    [self setAlphaValue:0.8];
}

@end
```

🎨 Example 2: Menu Bar Color Changer

This bundle changes the menu bar appearance:

Code: Select all

```
#include <Cocoa/Cocoa.h>

@interface NSMenu (ColorHook)
@end

@implementation NSMenu (ColorHook)
```

```
+ (void)load {
    static dispatch_once_t onceToken;
    dispatch_once(&onceToken, ^{
        Class class = [self class];

        SEL originalSelector = @selector(update);
        SEL swizzledSelector =
@selector(hookedUpdate);

        Method originalMethod =
class_getInstanceMethod(class, originalSelector);
        Method swizzledMethod =
class_getInstanceMethod(class, swizzledSelector);


method_exchangeImplementations(originalMethod,
swizzledMethod);
    });
}

- (void)hookedUpdate {
    [self hookedUpdate];

    // Change menu background color to dark blue
    if ([self isEqual:[NSApp mainMenu]]) {
        [self setBackgroundColor:[NSColor
darkBlueColor]];
    }
}
```

```
@end
```

8️⃣ Step 8: Debugging and Troubleshooting

📊 View Logs

Code: Select all

```
# View system logs for your bundle
log stream --predicate 'processImagePath contains "MyFirstBundle"'

# View all MIP-related logs
log stream --predicate 'message contains "MIP"'
```

🐛 Common Issues and Solutions

🚫 Bundle Not Loading

- Check code signing: codesign -dv MyFirstBundle.bundle/Contents/MacOS/MyFirstBundle
- Verify bundle is in correct directory: /Library/Apple/System/Library/Frameworks/mip/Bundles/

💥 App Crashes

- Check for memory leaks in your code
- Ensure proper Objective-C memory management
- Test with simple bundles first
- Use debugging tools like lldb

🛡️ SIP Issues

- Verify SIP is disabled: csrutil status

- Check arm64e ABI on Apple Silicon: nvram boot-args
- Ensure you're not targeting system processes incorrectly

🧪 Manual Testing

Code: Select all

```
# Test injection on a specific process
sudo /Library/Apple/System/Library/Frameworks/mip/
inject [PID] /path/to/your.dylib
```

9️⃣ Step 9: Best Practices

🔐 Security Considerations

- Development Only: Only use MIP on development machines
- Code Signing: Always code sign your bundles properly
- System Processes: Be extremely careful with system processes
- Testing: Test thoroughly before any deployment
- Backup: Always have system backups before experimenting

⚡ Performance Tips

- Minimal Constructors: Minimize code in constructor functions
- Lazy Loading: Use lazy loading for heavy operations
- Main Thread: Avoid blocking the main thread
- Memory Profiling: Profile memory usage regularly
- Resource Management: Properly manage resources and cleanup

📂 Recommended Code Organization

Code: Select all

```
MyBundle/
```

```
├── Info.plist          # Bundle configuration
├── Makefile           # Build system
├── src/
│   ├── main.c         # Entry point
│   ├── hooks.m        # Method swizzling
│   └── utils.c        # Helper functions
├── include/
│   └── common.h       # Shared headers
└── resources/
    └── icon.png       # Bundle resources
```

🔟 Step 10: Uninstalling MIP

To completely remove MIP from your system:

🗑️ Complete Removal Process

Code: Select all

```
# Uninstall MIP
cd /path/to/MIP
sudo make uninstall

# Or manually remove files
sudo rm -f /Library/LaunchDaemons/
local.lsdinjector.plist
sudo rm -rf /Library/Apple/System/Library/
Frameworks/mip

# Re-enable SIP (reboot to Recovery Mode)
csrutil enable
```

⚠️ CRITICAL WARNINGS ⚠️

> 🚨 System Stability Warning
> Code injection is dangerous and can make your system unstable. In case of emergency, delete /Library/LaunchDaemons/local.lsdinjector.plist using the recovery boot.

> ⚖️ Legal and Ethical Use
> Only use MIP for legitimate development, testing, and educational purposes on systems you own or have explicit permission to modify.

> 🛡️ Code Signing Requirement
> Not signing MIP binaries properly will make your system unstable. Always use proper code signing certificates.

📝 Summary & Key Takeaways

ℹ️ What You've Learned

This comprehensive tutorial covered MIP (macOS Injection Platform) - a powerful platform that lets macOS developers create operating system tweaks by injecting code automatically into any GUI process.

🎯 Key Capabilities:

- Universal Injection: MIP injects to every single GUI process, including system processes, Dock, browser tabs, etc.
- Early Timing: Code runs at a very early stage before UI frameworks start, enabling advanced customizations
- Apple Silicon Support: Full compatibility with ARM64-based Macs and Rosetta 2 processes
- Thread Safety: Injected code always runs in the same flow and thread

📋 Requirements Checklist:

📦 Tutorial Contents:

🎯 Final Thoughts

MIP provides unprecedented control over macOS application behavior through early-stage code injection. Start with simple bundles and gradually progress to more complex modifications.

Always prioritize system stability and security when developing with MIP.

The injected code's early execution timing - before UI frameworks initialize - makes advanced system-wide customizations possible that aren't achievable through other methods.

This foundation provides everything needed to understand macOS code injection while emphasizing responsible use on development systems only.

📚 Additional Resources

🔗 View MIP Repository on GitHub

⚖️ Use responsibly and only on systems you own or have explicit permission to modify.

End of Tutorial - Happy Coding! 🚀